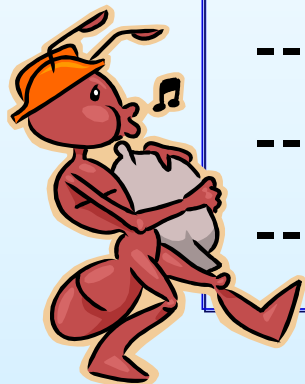


系统建模—4. 类图、对象图、构件图和部署图

• 学习目的和要求

- 类图的用途—**掌握**
- 类图的基本元素—**掌握**
- 阅读类图—**掌握**
- 构建类图—**掌握**



系统建模—3.类图模型

1. 类图的用途

在UML中通常建议使用“类图”作为表达领域模型的图形。

类图主要表达的是问题领域的“抽象概念”，在这个抽象概念中，除了表达该抽象概念的名称外，类（属性、操作）、协作（类或对象之间的协作）、接口及其关系。

□ “类图”的用途

- 在系统分析阶段，类图主要用于显示角色和提供系统行为的实体的职责；
- 在系统设计阶段，类图主要用于捕获组成系统体系结构的类结构；
- 在系统编码阶段，根据类图中的类及他们之间的关系实现系统的功能。

系统建模—3.类图模型

1.公共 (public) : 公共成员可以被任何类访问。这种成员通常用于提供公共接口, 使外部用户可以访问和操作类的状态和行为。

2.私有 (private) : 私有成员只能被定义它们的类所访问。这种成员通常用于封装类的内部实现细节, 以确保外部用户无法直接修改类的状态。

3.受保护 (protected) : 受保护成员可以被定义它们的类及其子类访问。这种成员通常用于允许子类重写或访问父类的行为。

4.包级私有 (package-private) : 包级私有成员只能被定义它们的包中的类访问。这种成员通常用于实现类的内部协作, 并且不希望对包外的类暴露实现细节。

➤ 类的属性:

- ✓ 描述了类的静态特征; 属性名的第一个字母小写;
- ✓ 属性的定义格式:

[可见性] 属性名 [: 类型] [' [' 多重性 [次序] ']] [=初始值] [{特性}]

+、-、#、~

系统建模—3.类图模型

1.公共 (public) : 公共成员可以被任何类访问。这种成员通常用于提供公共接口, 使外部用户可以访问和操作类的状态和行为。

2.私有 (private) : 私有成员只能被定义它们的类所访问。这种成员通常用于封装类的内部实现细节, 以确保外部用户无法直接修改类的状态。

3.受保护 (protected) : 受保护成员可以被成员通常用于允许子类重写或访问父类的行为。

4.包级私有 (package-private) : 包级私有访问。这种成员通常用于实现类的内部协作, 并

➤ 例:

```
#visibility:Boolean=false  
colors:Color[3]  
points:Point[2..* ordered]  
name:String[0..1]
```

➤ 类的属性:

- ✓ 描述了类的静态特征; 属性名的第一个字母小写;
- ✓ 属性的定义格式:

[可见性] 属性名 [: 类型] [' [' 多重性 [次序] ']] [=初始值] [{特性}]

+、-、#、~

系统建模—3.类图模型

2. 类图的基本元素

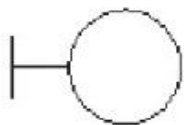
➤ 类的操作：

- ✓ 操作名的命名规范习惯采用和属性名相同的命名规则。
- ✓ 类的操作定义格式：

[可见性]操作名[(参数列表)][:返回类型][{特性}]

➤ 类的职责：

- ✓ 指类承担的责任和义务，在矩形框的最后一栏中写明类的职责



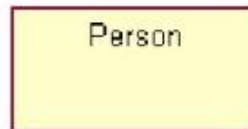
边界类(图标)



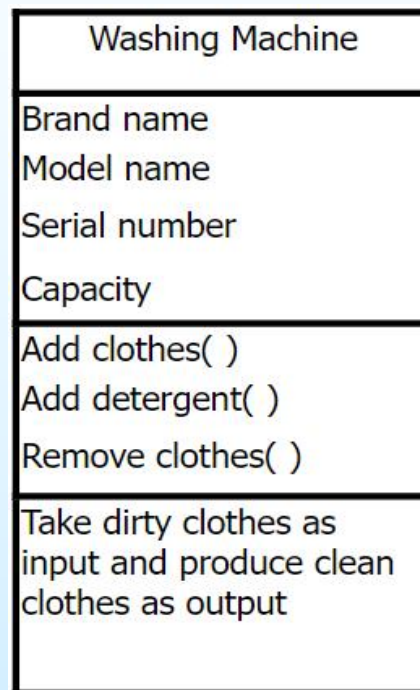
实体类(图标)



控制类(图标)



类的缩略表示



系统建模—3.类图模型

2. 类图的基本元素

➤ 类的操作：

- ✓ 操作名的命名规范习惯采用和
- ✓ 类的操作定义格式：

[可见性]操作名[(参数列表)][[:返回类型]][[特性]]

➤ 例：

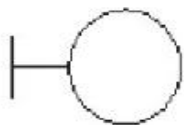
+hide():Boolean

#create()

-attachXWindow(xwin:XwindowPtr)

➤ 类的职责：

- ✓ 指类承担的责任和义务，在矩形框的最后一栏中写明类的职责



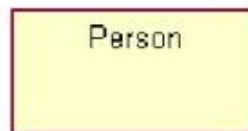
边界类(图标)



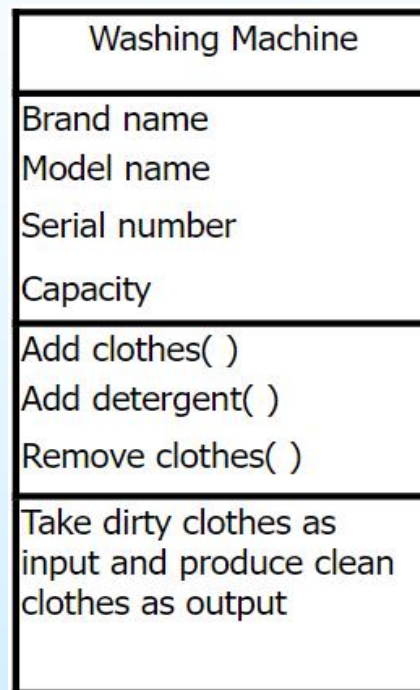
实体类(图标)



控制类(图标)



类的缩略表示



系统建模—3.类图模型

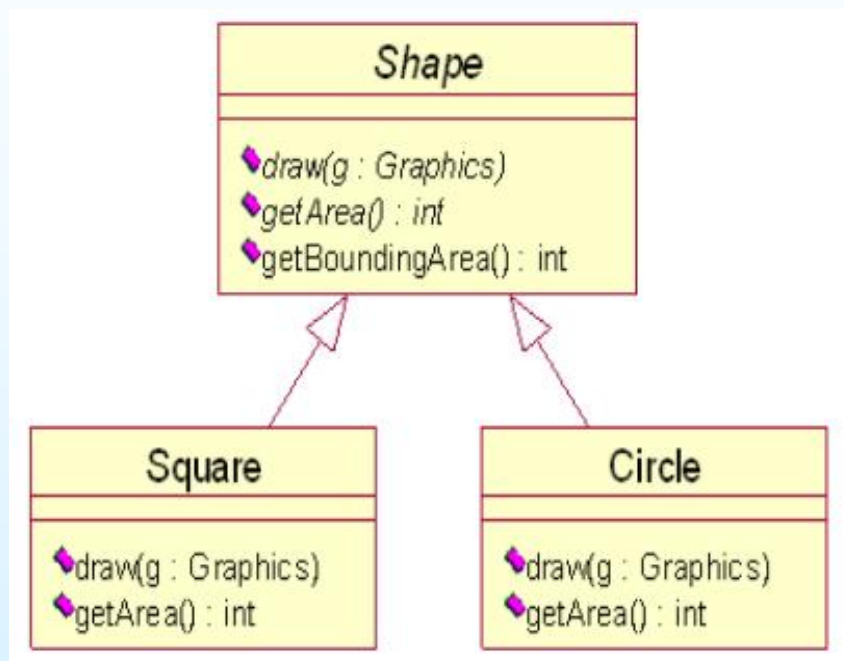
2. 类图的基本元素

类的种类

1) 抽象类

- 当某些类有一些共性的方法或属性时，可以定义一个抽象类来抽取这些共性，然后将包含这些共性方法和属性的具体类作为该抽象类的继承；
- 抽象类是一种不能直接实例化的类，不能用抽象类创建对象；
- 抽象类可以实现多态。

在UML中，抽象类和抽象方法的表示是将其名字用**斜体**表示：

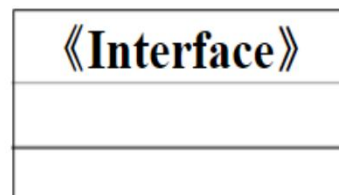


系统建模—3.类图模型

2. 类图的基本元素

2) 接口

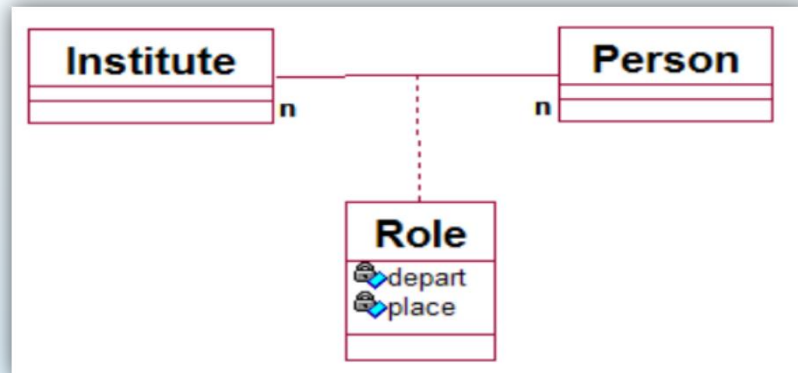
- 接口是一种类似于抽象类的机制，是一个没有具体实现的类
- 接口可以实现多态



构造符号表示法

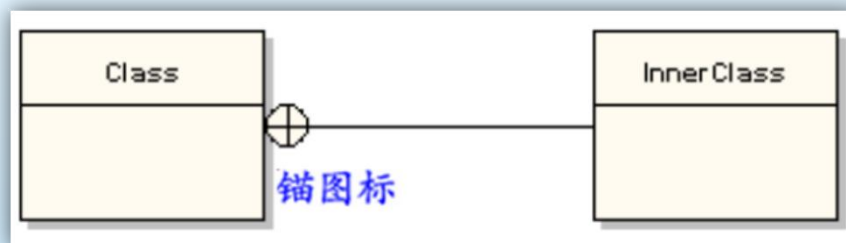
3) 关联类

- 当两个类之间具有多对多的关系，并且有些属性不属于关联两端任何一个类
- 关联类通过虚线与关联连接



4) 嵌套类

- 将一个类的定义放在另一个类定义的内部，就是嵌套类



系统建模—3.类图模型

2. 类图的基本元素

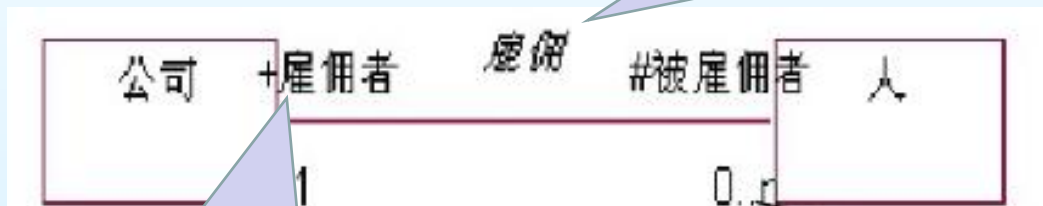
(2) 关联 (Association) :

关联是模型元素间的一种语义联系，当类之间在概念上有连接关系时，其间的连接叫做关联。关联有名称、角色、多重性和导航性等语法。

➤角色

- ✓ 关联两端的类可以某种角色参与关联
- ✓ 使用名词或名词短语

关联名：如果关联关系已经清楚，就无需关联名



角色名：如果关联名与角色名相同则不标

关联的多重性

➤关联名

- ✓ 描述关联的作用
- ✓ 使用动词或动词短语

系统建模—3.类图模型

2. 类图的基本元素

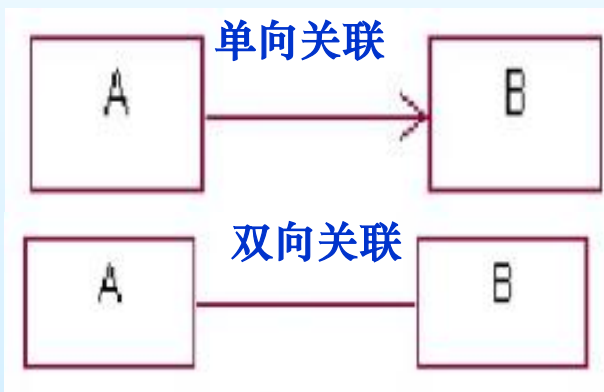
➤多重性

- ✓ 某个类有多个对象可以和另一个类的单个对象关联
- ✓ 在UML中，常用的关联的多重性表示

➤ 0..1	0或1
➤ 1	1
➤ 0..*(0..n)	0或多个
➤ *	0或多个
➤ 1..*(1..n)	1个或多个
➤ 8	8
➤ 5, 7..10	5或7~10

➤导航性

- 用箭头表示导航性。描述源对象通过链接访问目标对象，A类对象存储B类对象的一个引用，也就是说由A类可以找到B类

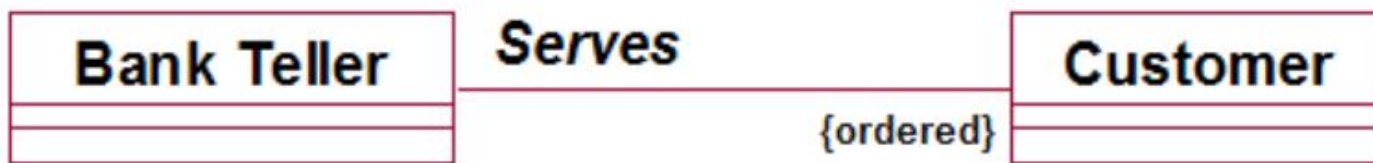


系统建模—3.类图模型

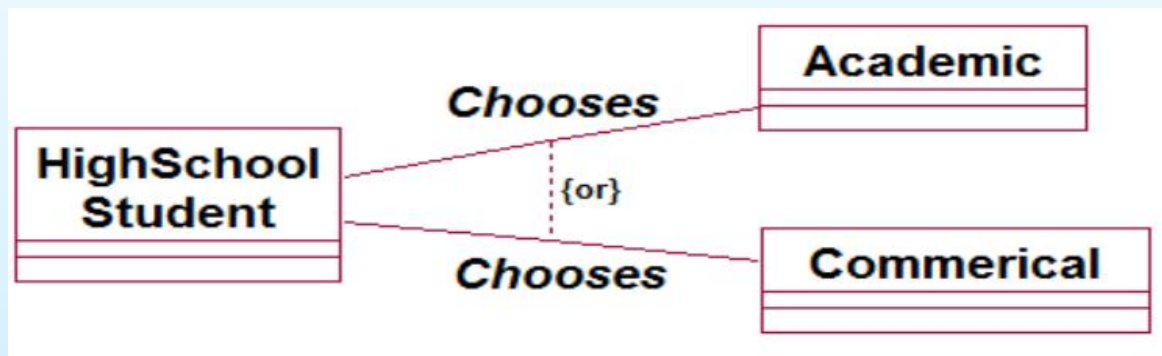
2. 类图的基本元素

➤ **关联的约束** 关联可加上一些约束，以加强关联的含义

例1： 一个BankTeller为顾客服务，但是服务的顺序要按照顾客排队的次序进行。在模型中可以在Customer类附近加上一个花括号括起来的“ordered（有序）”来说明这个规则，也就是知名约束。



例2： 高中生选专业课或选商务课

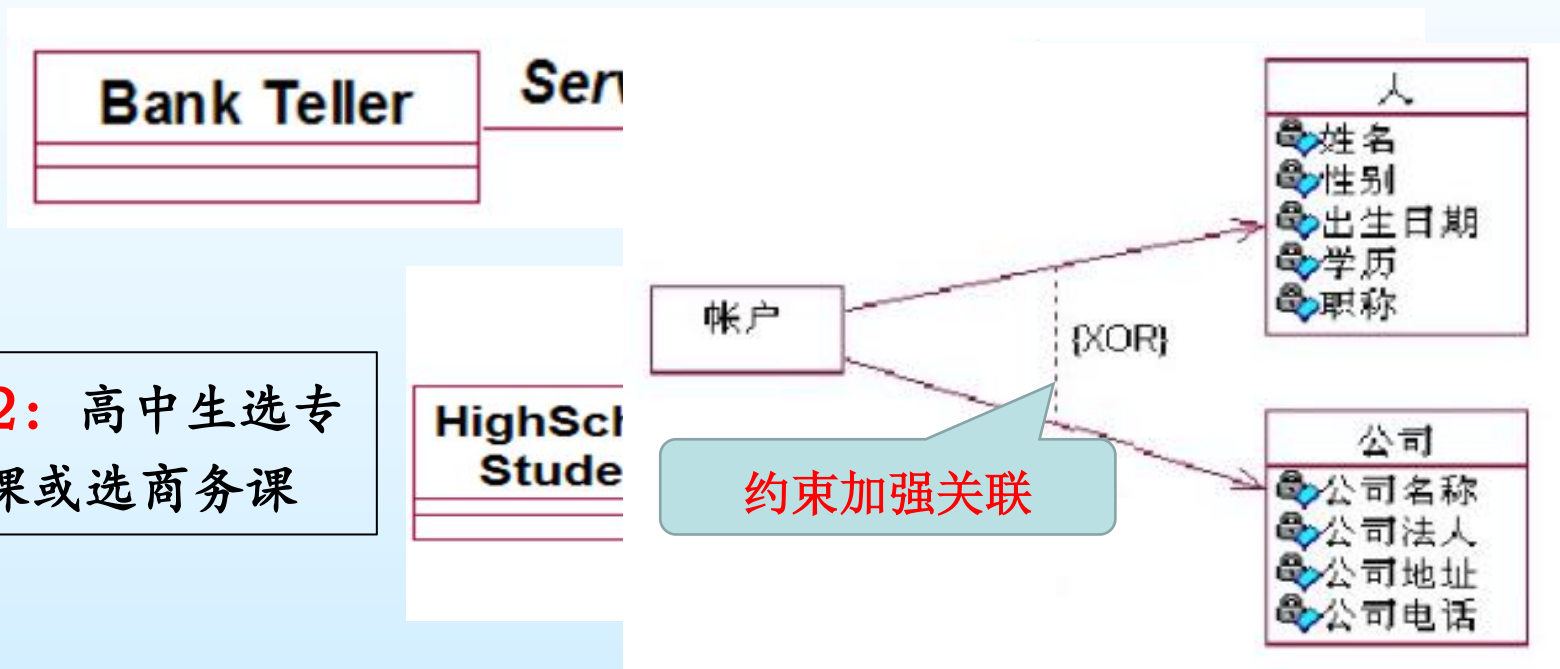


系统建模—3.类图模型

2. 类图的基本元素

➤ **关联的约束** 关联可加上一些约束，以加强关联的含义

例1： 一个BankTeller为顾客服务，但是服务的顺序要按照顾客排队的次序进行。在模型中可以在Customer类附近加上一个花括号括起来的“ordered（有序）”来说明这个规则，也就是知名约束。



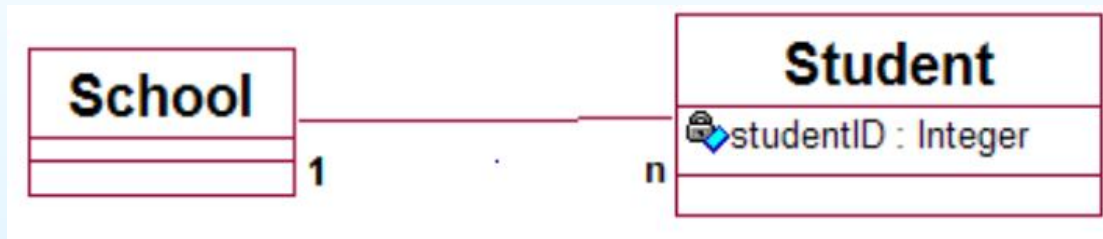
例2： 高中生选专业课或选商务课

系统建模—3.类图模型

2. 类图的基本元素

➤ 限定符

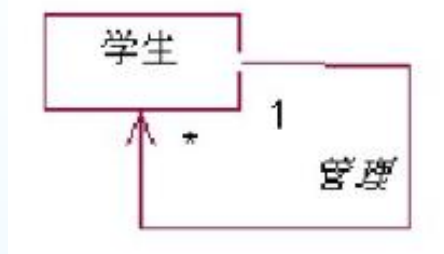
- ✓ 在关联端紧靠源类图标处可有限定符，带有限定符的关联称为限定关联。
- ✓ 限定符的作用：用于将一个多（或一）对多关联转化为一个多（或一）对一关联。
- ✓ 限定符是关联的属性，而不是类的属性



系统建模—3.类图模型

2. 类图的基本元素

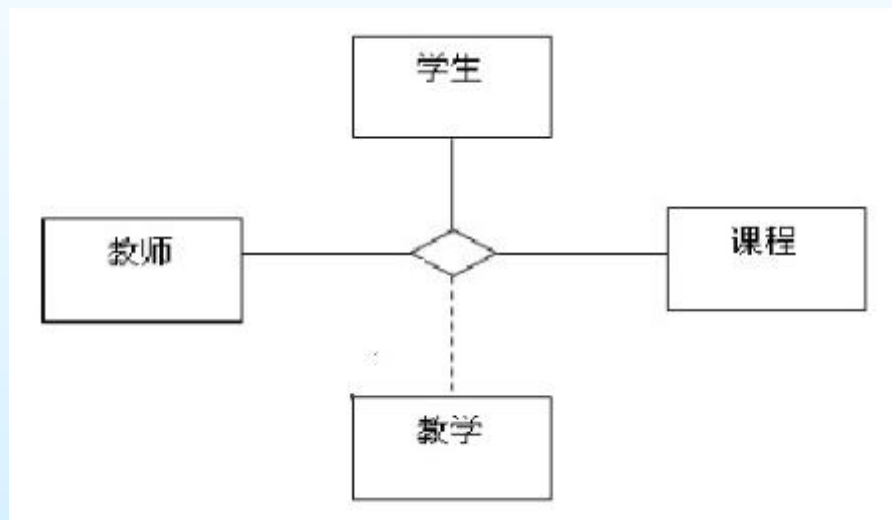
➤ 关联的分类



一元(自返)关联



二元关联

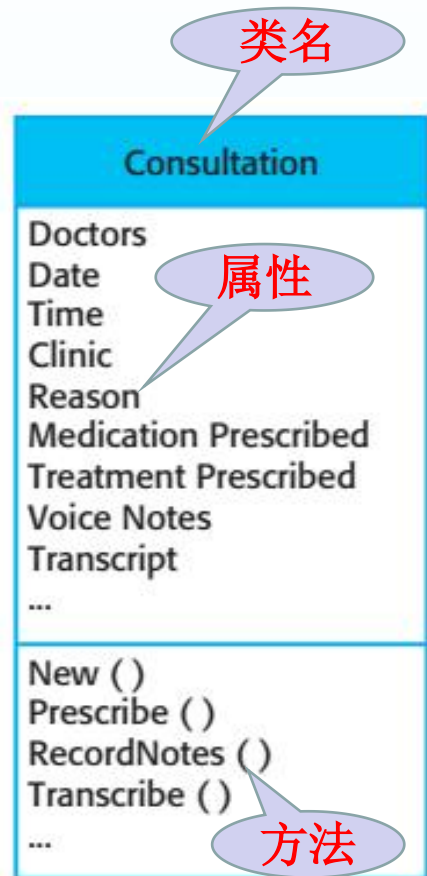
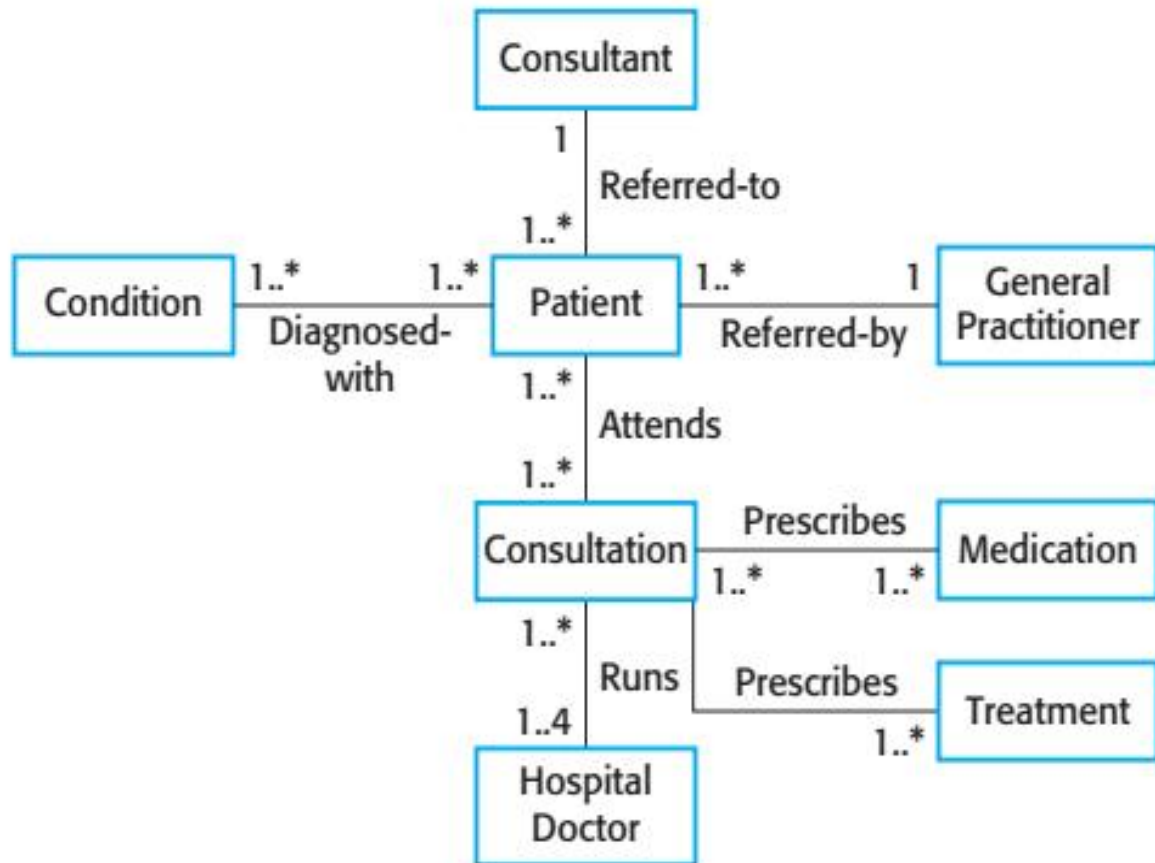


多元关联

系统建模—3.类图模型

2. 类图的基本元素

例：心理治疗信息管理系统类图



□ 诊断类

系统建模—3.类图模型

2. 类图的基本元素

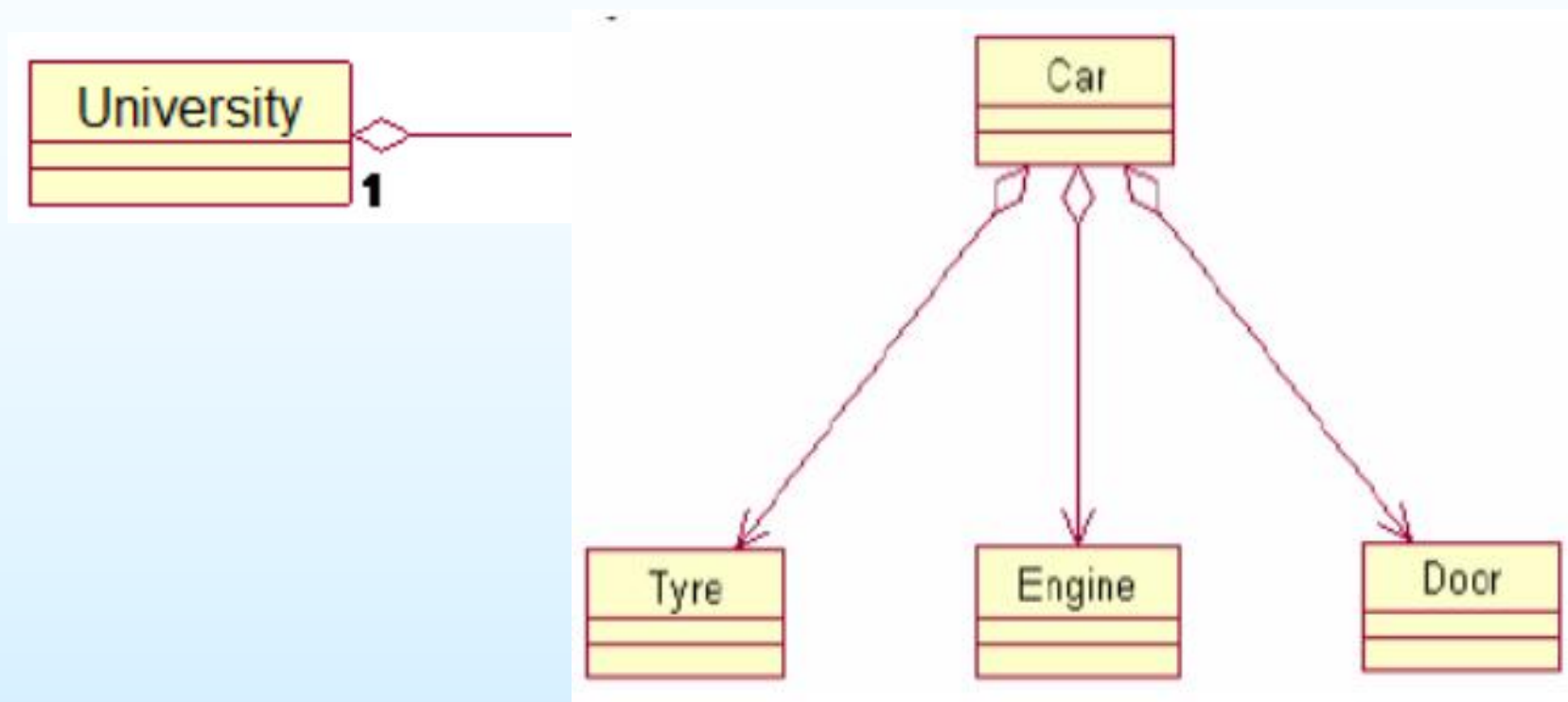
1) 聚集：表示类之间的一种松散的整体与部分的组成关系，是一种特殊的关联。整体类位于部分类的上层，多个部分类处于并列的层次。



系统建模—3.类图模型

2. 类图的基本元素

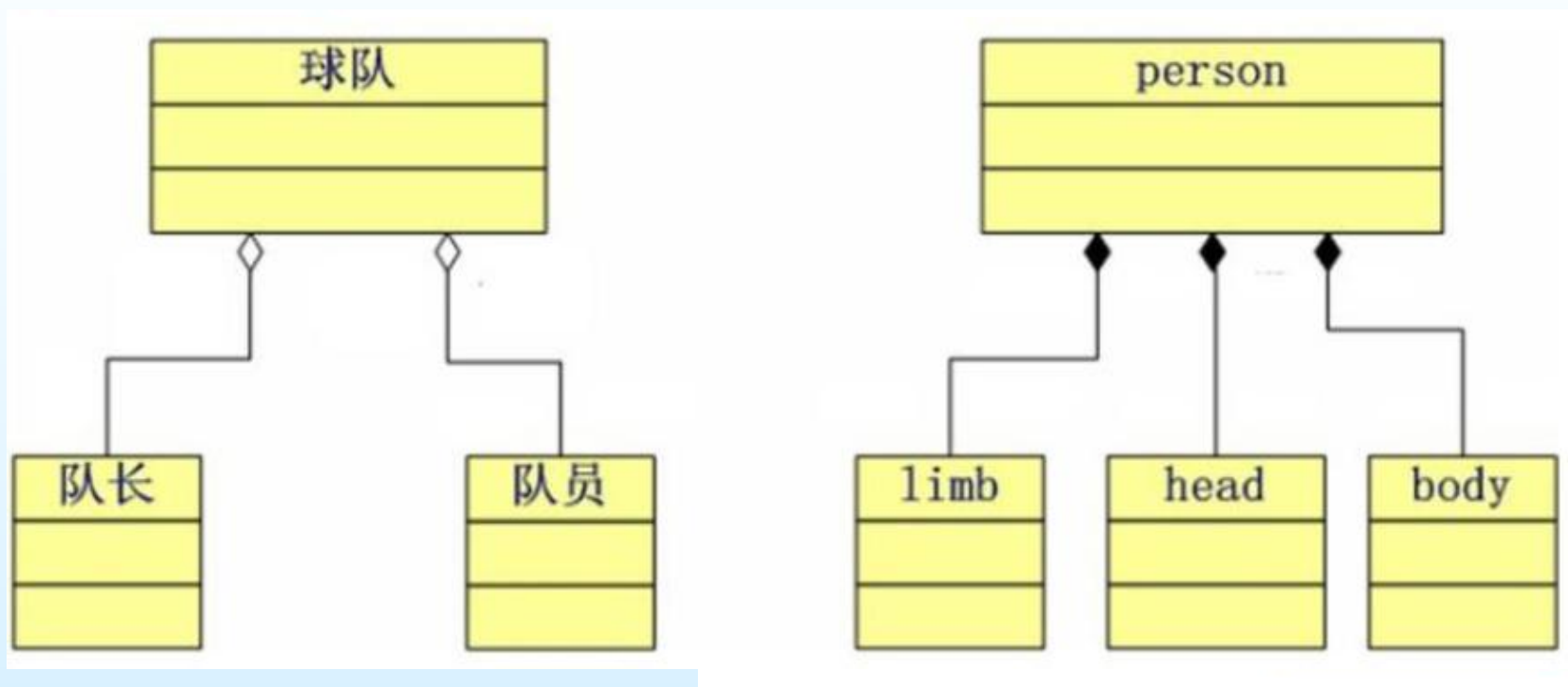
1) 聚集：表示类之间的一种松散的整体与部分的组成关系，是一种特殊的关联。整体类位于部分类的上层，多个部分类处于并列的层次。



系统建模—3.类图模型

2. 类图的基本元素

2) 组合：是一种更强形式的聚合（强聚合），聚合中的每个部分只能属于一个整体整体和部分具有相同的生命周期。

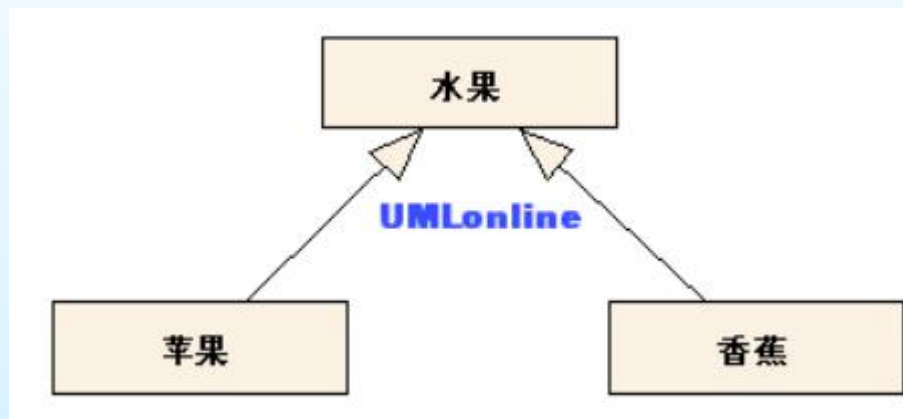
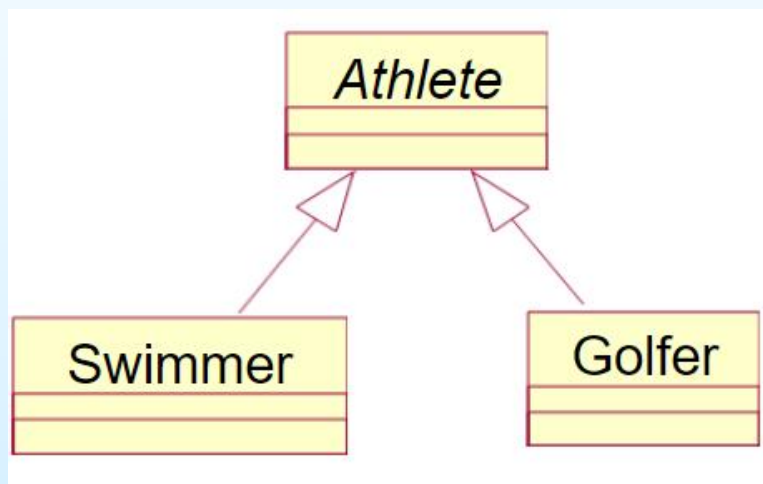


系统建模—3.类图模型

2. 类图的基本元素

3) 泛化：表示事物之间的一般与特殊的关系，也称为继承关系。UML中有专门的关联类型来表示泛化。泛化用指向上面更一般的类的箭头表示。

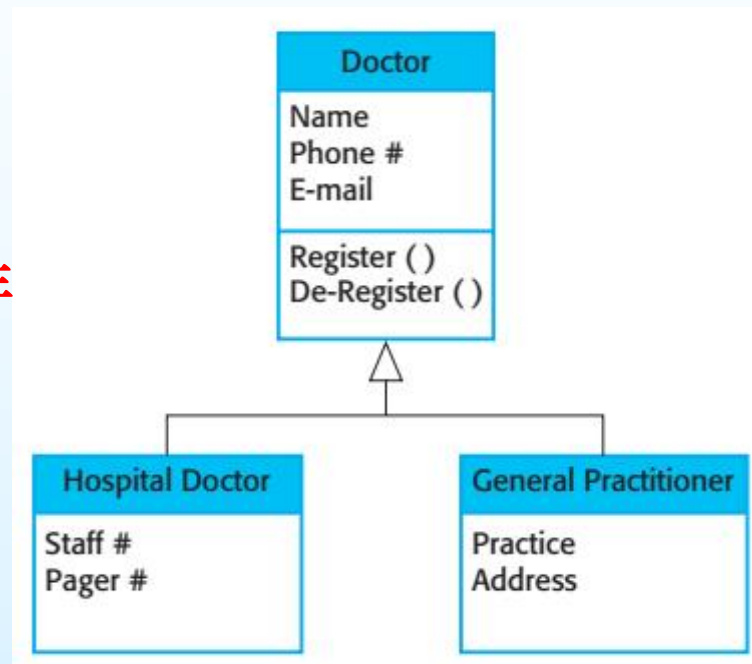
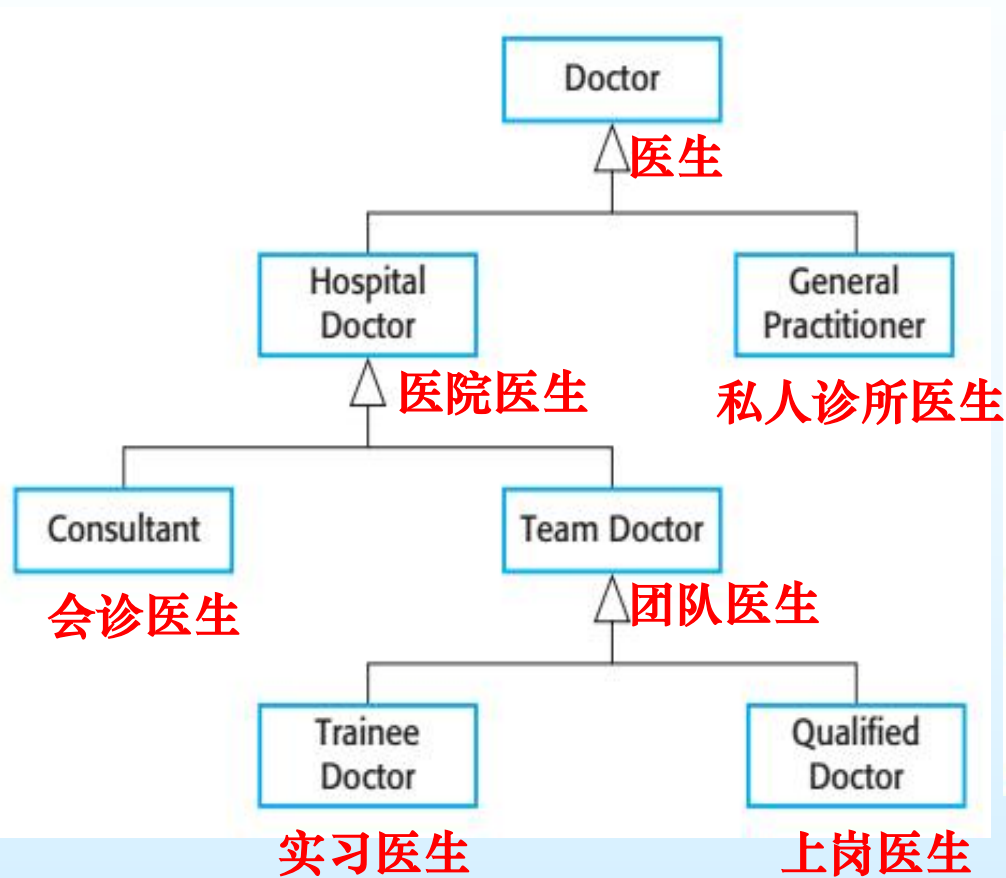
在泛化关系中，更高层次上类中的属性和操作也是较低层次上的类的属性和操作。简言之，低层的是子类，它继承了超类的属性和操作。



系统建模—3.类图模型

2. 类图的基本元素

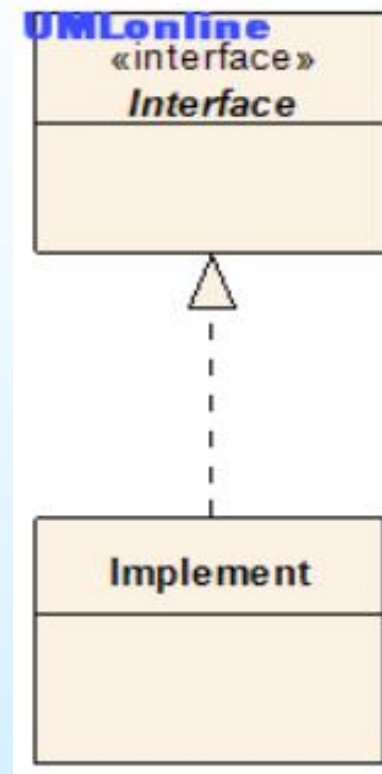
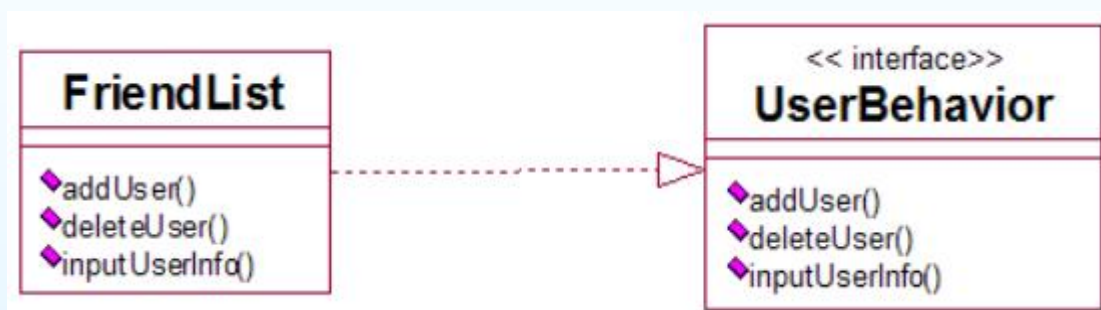
□例子：泛化关系



系统建模—3.类图模型

2. 类图的基本元素

4) **实现：**表达一种说明元素与实现元素之间的关系；类和接口之间的关系是实现关系，表示类实现接口提供的操作显示一个类引用另一个类。



系统建模—3.类图模型

2. 类图的基本元素

5) 依赖：两个元素之间的一种关系，其中一个元素（提供者）的变化将影响另一个元素（客体），或向它提供所需信息，显示一个类引用另一个类。

软件开发中，往往会设计一些共用类，供别的类调用，如果这些公用类出问题了，那调用这些公用类都会因此而出问题。

例：假设你正在设计一个能显示公司全体成员的制表系统，公司的员工可以填写这个系统中的电子表格。员工要选择菜单来填写表格。在你的设计中，有一个System（系统）类和一个Form（表格）类。System类的众多操作中有一个displayForm(f:Form)，系统所要显示的表格取决于用户选择的表格。

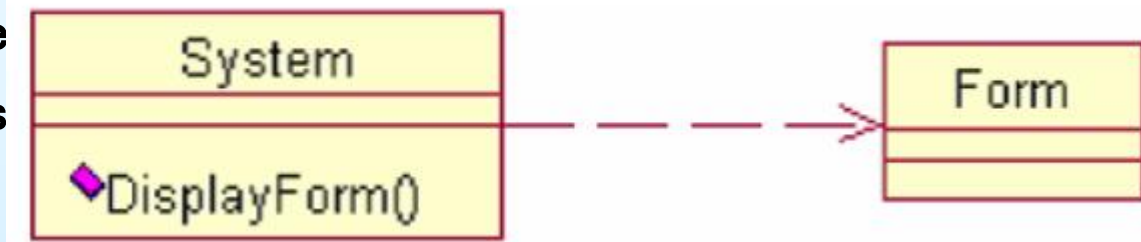
系统建模—3.类图模型

2. 类图的基本元素

5) 依赖：两个元素之间的一种关系，其中一个元素（提供者）的变化将影响另一个元素（客体），或向它提供所需信息，显示一个类引用另一个类。

软件开发中，往往会设计一些共用类，供别的类调用，如果这些公用类出问题了，那调用这些公用类都会因此而出问题。

例：假设你正在设计一个能显示公司全体成员的制表系统，公司的员工可以填写这个系统中的电子表格。员工要选择菜单来填写表格。在你的设计中，有一个System类，它有一个displayForm()方法。在你的设计中，有一个Form类，它有一个displayForm()方法。在你的设计中，有一个System类，它有一个displayForm()方法。在你的设计中，有一个Form类，它有一个displayForm()方法。



系统建模—3.类图模型

2. 类图的基本元素

依赖关系可以分为四类：使用、抽象、授权、绑定依赖。

① **使用依赖** 表示客户使用提供者提供的服务以实现它的行为，包括：

- **使用**<<use>>--声明使用一个类时需要用到已存在的另一个类。
- **调用**<<call>>--声明一个类调用其他类的操作的方法。
- **参数**<< parameter >>--声明一个操作和它的参数之间的关系。
- **发送**<< send>>--声明信号发送者和信号接收者之间的关系。
- **实例化**<<instantiate>>--声明用一个类的方法创建了另一个类的实例。

② **绑定依赖** 较高级的依赖类型，用于绑定模板以创建新的模型元素。

- **绑定**<<bind>>--为模板参数指定值，以生成一个新的模型元素。

系统建模—3.类图模型

2. 类图的基本元素

③ 抽象依赖

表示客户与提供者之间用不同的方法表现同一个概念，通常一个概念更抽象，一个概念更具体。包括：

- **跟踪**<<trace>>--声明不同模型中的元素之间存在一些连接但不如映射精确。
- **精化**<<refine>>--声明具有两个不同语义层次上的元素之间的映射。
- **派生**<<derive>>--声明一个实例可以从另一个实例导出。

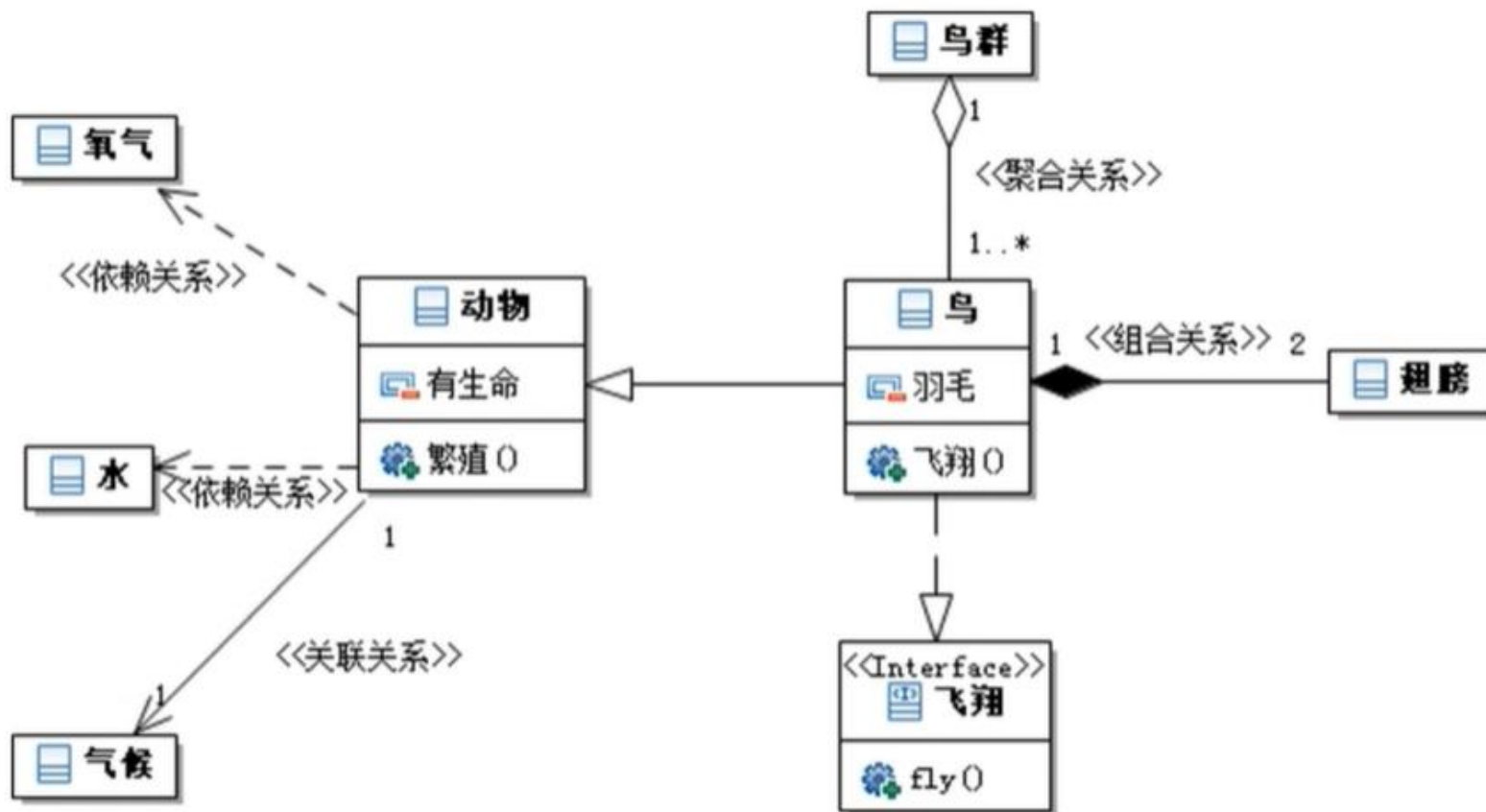
④ 授权依赖

- **访问**<<access>>--允许一个包访问另一个包的内容。
- **导入**<<import>>--允许一个包访问另一个包的内容并为被访问包的组成部分增加别名。
- **友元**<<friend>>--允许一个元素访问另一个元素，不管被访问的元素是否具有可见性。

3. 阅读类图

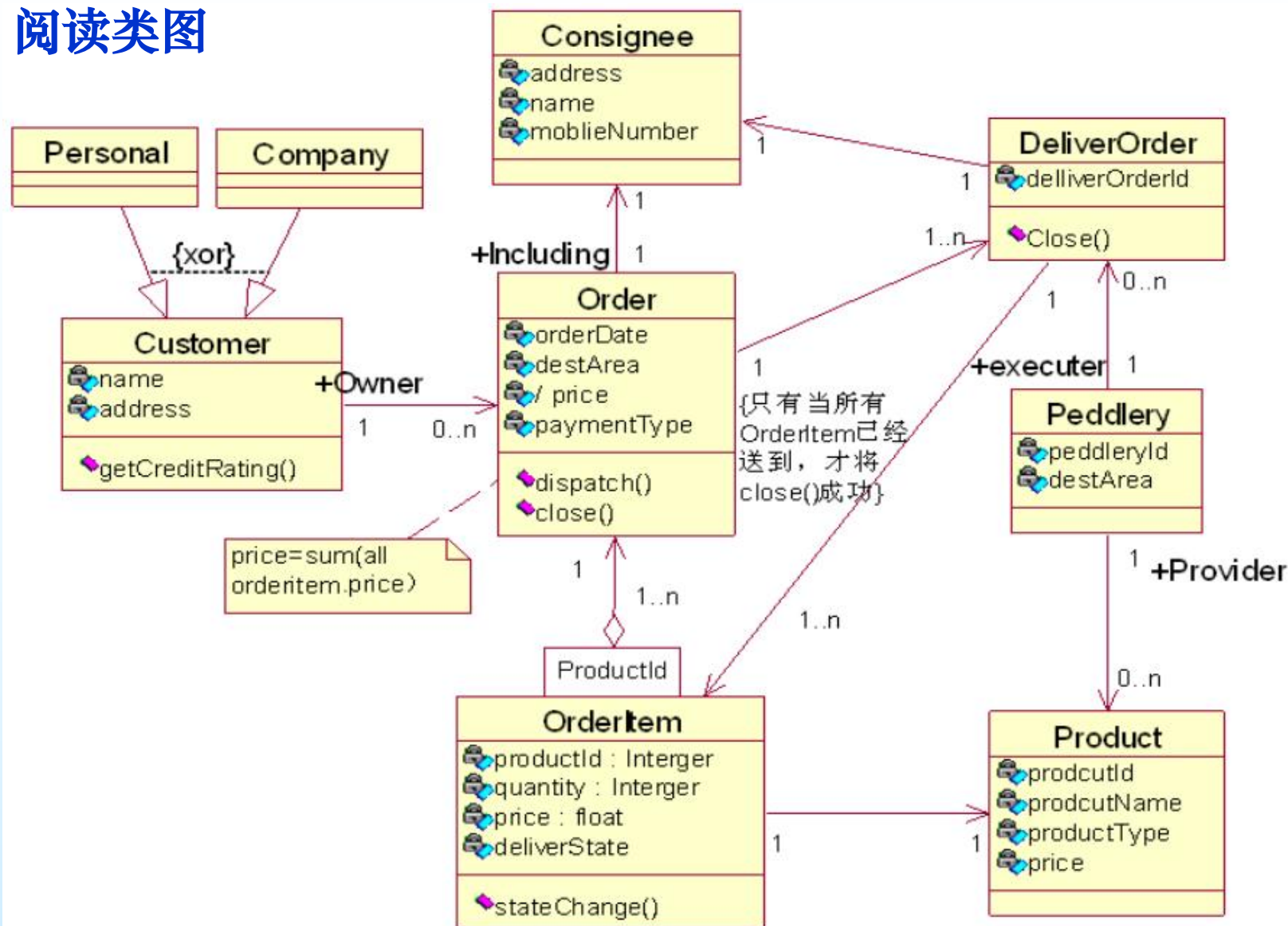
□ 例子：动物养殖类图

- 先看清有哪些类；
- 然后看看类之间存在的关系；
- 结合多重性来理解类图的结构特点以及各属性和方法的含义



系统建模—3.类图模型

3. 阅读类图

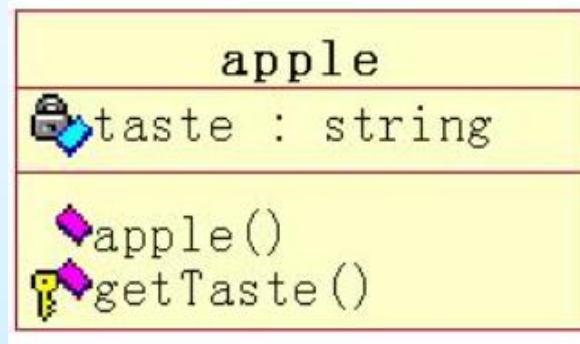
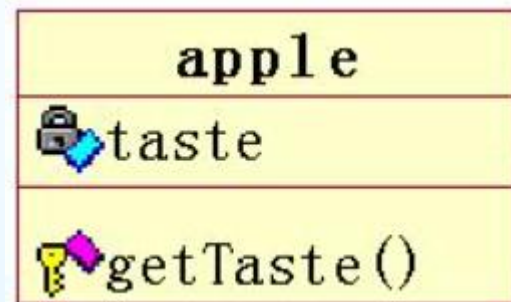


系统建模—3.类图模型

4、如何建立类图

(1) 类图的抽象层次

- **概念类**：描述应用领域中的概念，仅包含类名，不考虑细节。
- **分析类**：分析不针对具体语言，包含一些类的细节特性。
- **设计类**：这对具体的语言，考虑类的实现细节



系统建模—3.类图模型

4、如何建立类图

(2) 建立类图的步骤

① 分析问题域，确定需求

② 寻找类，确定类的含义和职责：在用例模型中寻找名词，包括角色，根据边界类、控制类、实体类帮助分析系统中的类。

③ 定义类的属性和操作

- 按一般常识，这个对象有哪些属性？当前问题领域，该对象有哪些属性？
- 根据系统责任（功能），这个对象应该增加哪些属性？
- 为了实现某些功能，对象需要增加哪些属性？
- 对象有哪些区别的状态？整体和部分属性设置（类型：如计算机类）

④ 确定类之间的结构关系

- 发现一般—特殊结构（泛化关系）
- 发现整理—部分结构（特殊关联：组合聚合）
- 发现其它关系（关联、依赖、实现）

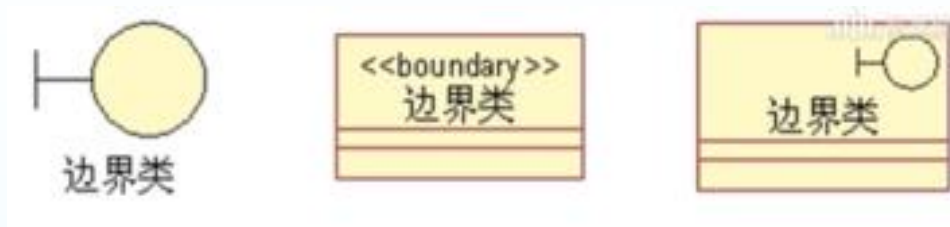
⑤ 精化类和类之间的关系

⑥ 绘制类图

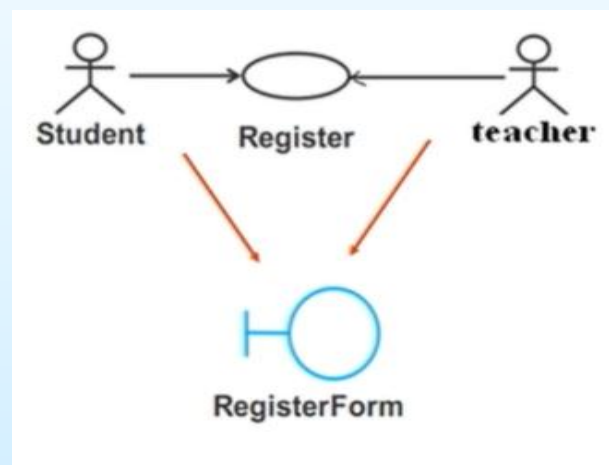
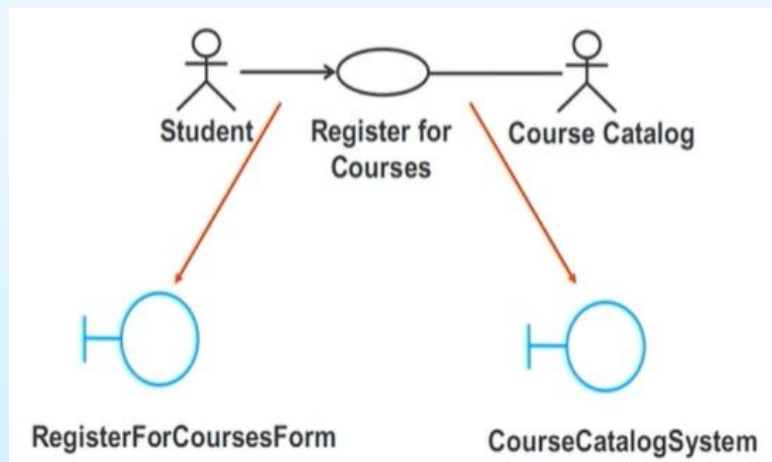
系统建模—3.类图模型

4、如何建立类图

(1) **边界类**：位于系统与外界
的交界处。如：窗体、对话框、报
表、以及表示通讯协议的类、直接
与外部设备交互的类。



通过用例图可以确定需要的边界类，每个**actor**对至少要有一个边界类。但并非每个**actor**对要生成唯一的边界类，**例如：多个actor启动一个用例时，可以用一个边界类与系统通信。**



系统建模—3.类图模型

4、如何建立类图

- **用户界面类**：帮助与系统用户进行通信的类，通过标准I/O设备提供人机界面。
- **系统接口类**：帮助与其他系统进行通信的类，系统接口对象隐藏如何与外部接口通信的细节。
 - 例如ATM系统中，提款必须通过ATM网络得到验证，然后该网络再通过银行会计系统对提款进行验证，我们可以确定一个ATM网络接口，让其对象来提供与ATM网络之间的通信。
- **设备接口类**：为用来监测外部事件的（硬件）设备（如传感器、Timer）提供接口的类。

从架构角度讲，边界类主要位于展现层（用户界面层）！

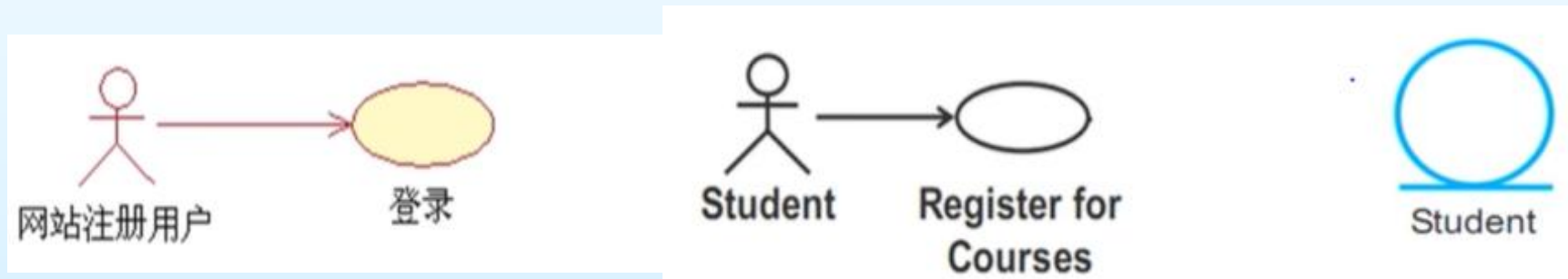
系统建模—3.类图模型

4、如何建立类图

(2) 实体类：用于对必须存储的信息和行为建模的类。持久存储体就是数据库、文件等可以永久存储数据的介质。



通常每个实体类在数据库中有相应的表，实体类中属性对应数据库表中的字段。但这并不意味着，实体类和数据库中的表是一一对应的。有可能是一个实体类对应多个表，也有可能是多个实体类对应一个表。



从架构的角度上来说，实体类主要位于数据持久层(数据层)！

系统建模—3.类图模型

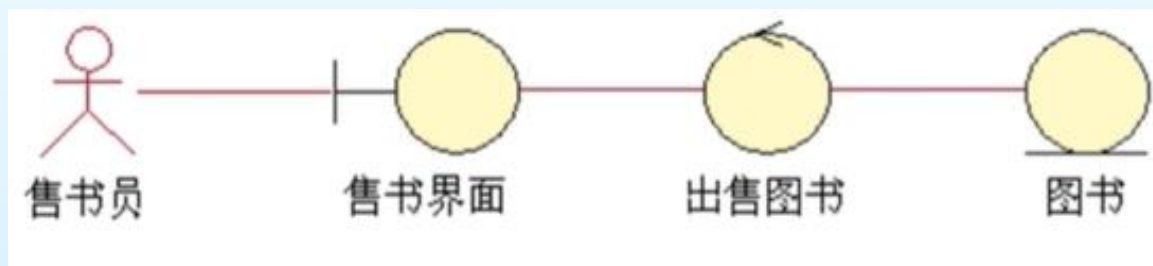
4、如何建立类图

(3) **控制类**：用于对一个或几个用例所特有的控制行为进行建模。



控制类对象通常被看成一个乐队的指挥，他指挥（控制）参与**use case**的其他对象的行为，通过对象什么时候执行以及执行什么，完成功能实现的总体逻辑流程。

其它类并不向控制类发送很多消息，而是控制类向其他类对象发出很多消息。



系统建模—3.类图模型

4、如何建立类图

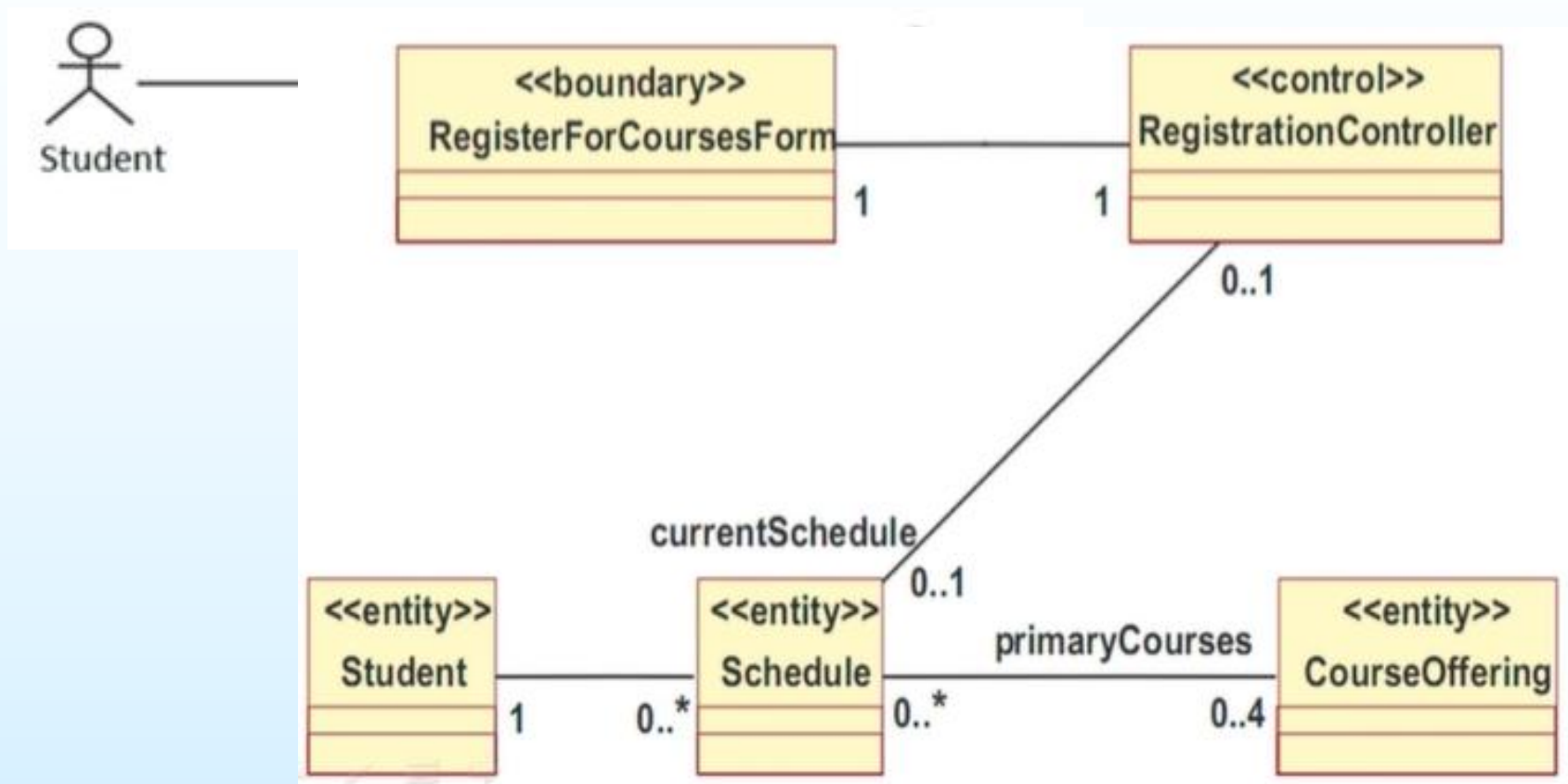
□ 例1：学生选课系统类图构建



系统建模—3.类图模型

4、如何建立类图

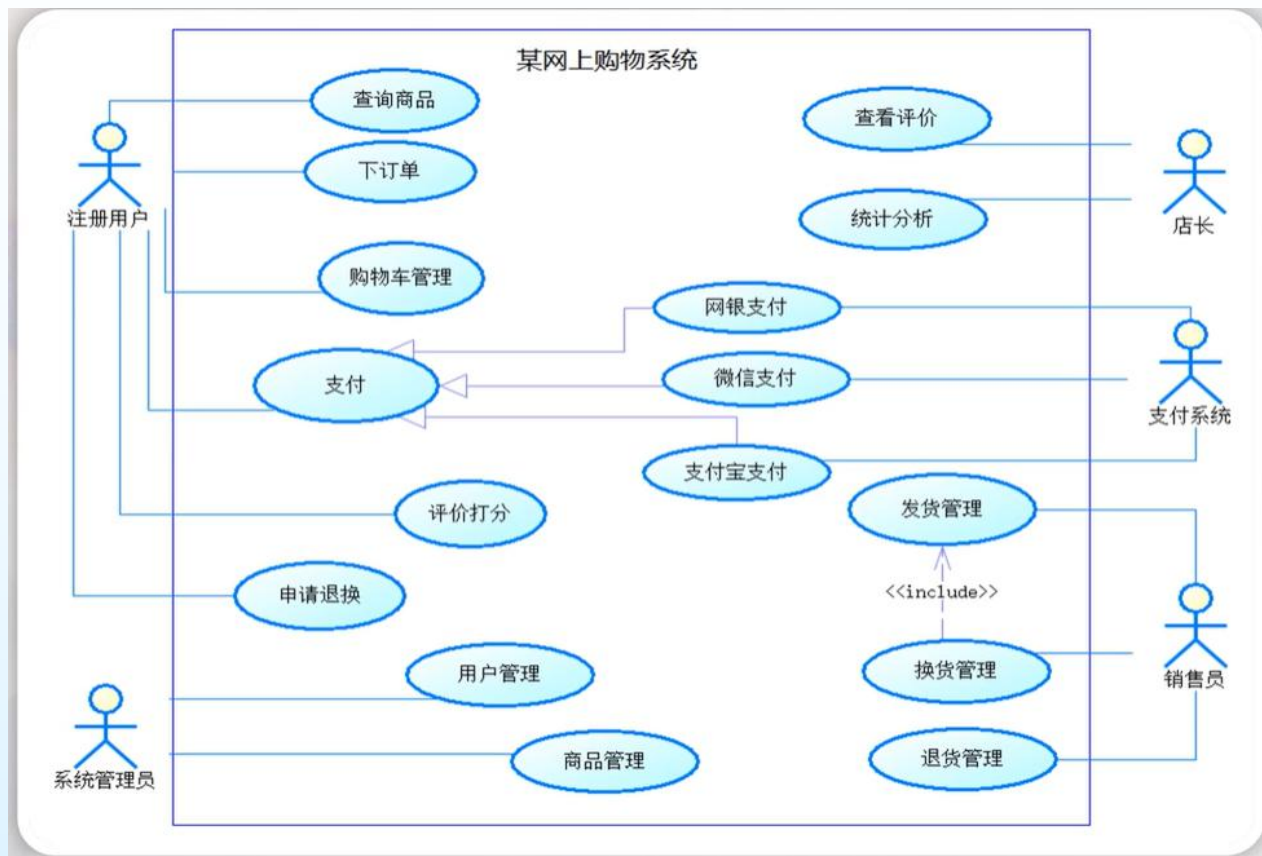
□ 例1：学生选课系统类图构建



系统建模—3.类图模型

4、如何建立类图

□ 例2：网上购物系统类图构建



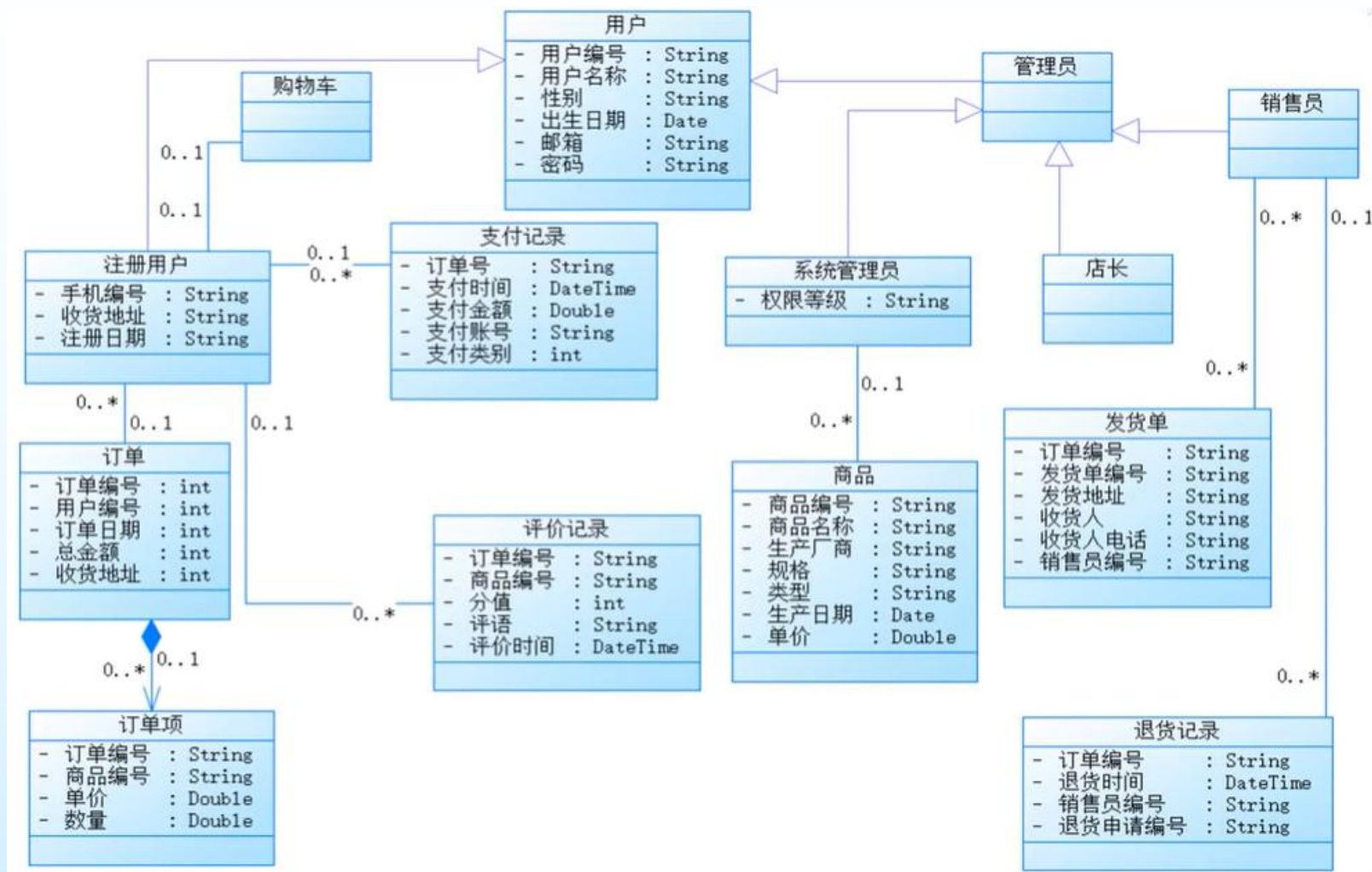
- 寻找得到的结果：
注册用户、销售员、系统管理员、店长
- 寻找要保存的结果：
- 申请退还->退还申请
- 支付->支付记录
- 评价打分->评价结果
- 发货->发货单
-

- 注册用户、销售员、系统管理员、店长、商品
- 订单、购物车、用户、退换申请、支付记录、评价结果、发货单

系统建模—3.类图模型

4、如何建立类图

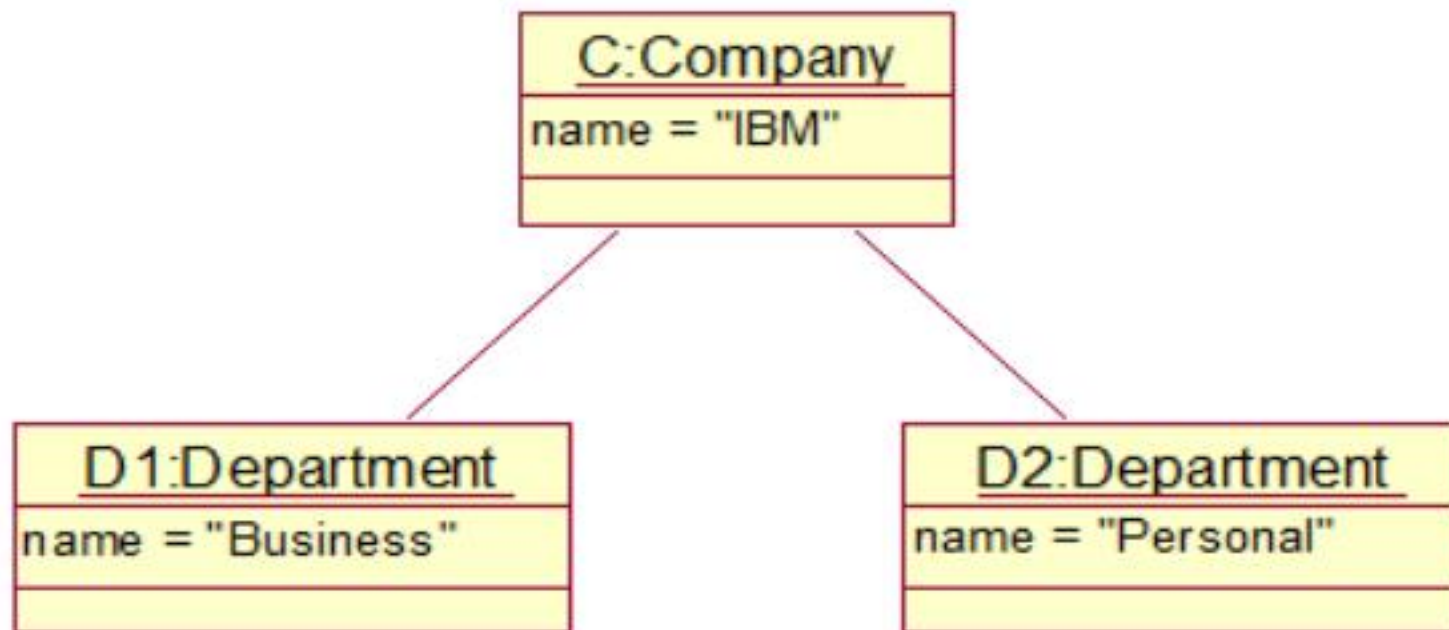
□ 例2：网上购物系统类图构建



系统建模—4.对象图模型

对象图表示一组对象之间联系，对象图是类图的实例。

- 对象名：在矩形框的顶端显示。



类图和对象图是建立对象模型主要工具，用于各类系统：信息管理系统、数据库系统、Web应用系统、实时控制系统。

系统建模—5.包

- UML中包是对模型元素进行组织管理的通用机制。包内可以包含UML中任何元素，如类、用例、接口、组件、节点等，也可以包含其它包、用例图、协作图、序列图等。
- 把语言相近，可能一起变更模型元素组织在包里，便于理解复杂系统。
- 包图由包和包间联系构成，包的联系：**依赖**、**泛化**。
- **包依赖：**
一个元素定义改变引起另一元素发生相应改变，用虚线箭头表示包间依赖关系，虚箭线从依赖包指向独立包。
- **包泛化：**
两个包间有一般特殊关系，实线箭头表示包间泛化关系。

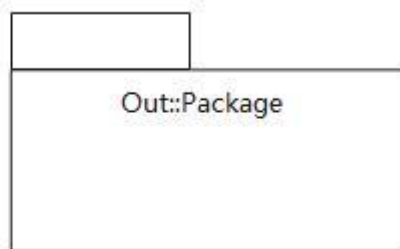
系统建模—5.包图

包名：每个包必须有一个与其他包相区别的名称，包的名字是一个字符串，有简单名和路径名之分。

•简单名：仅含一个简单的名称。 路径名：以包所位于的外围包的名字作为前缀的包名，其基本语法是：外部包名称::本包名称。



简单名

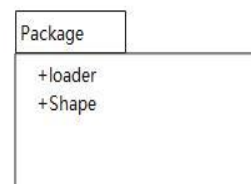


路径名

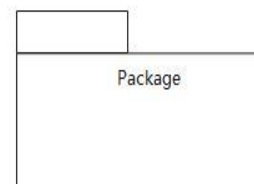


“Borrow”这个包位于 “SystemInterface”包内。

•当不需要显示包的内容时，将包的名字放入主方框内； 当需要显示包的内容时，将包的名字放入左上角的小方框中，内容放入主方框内。

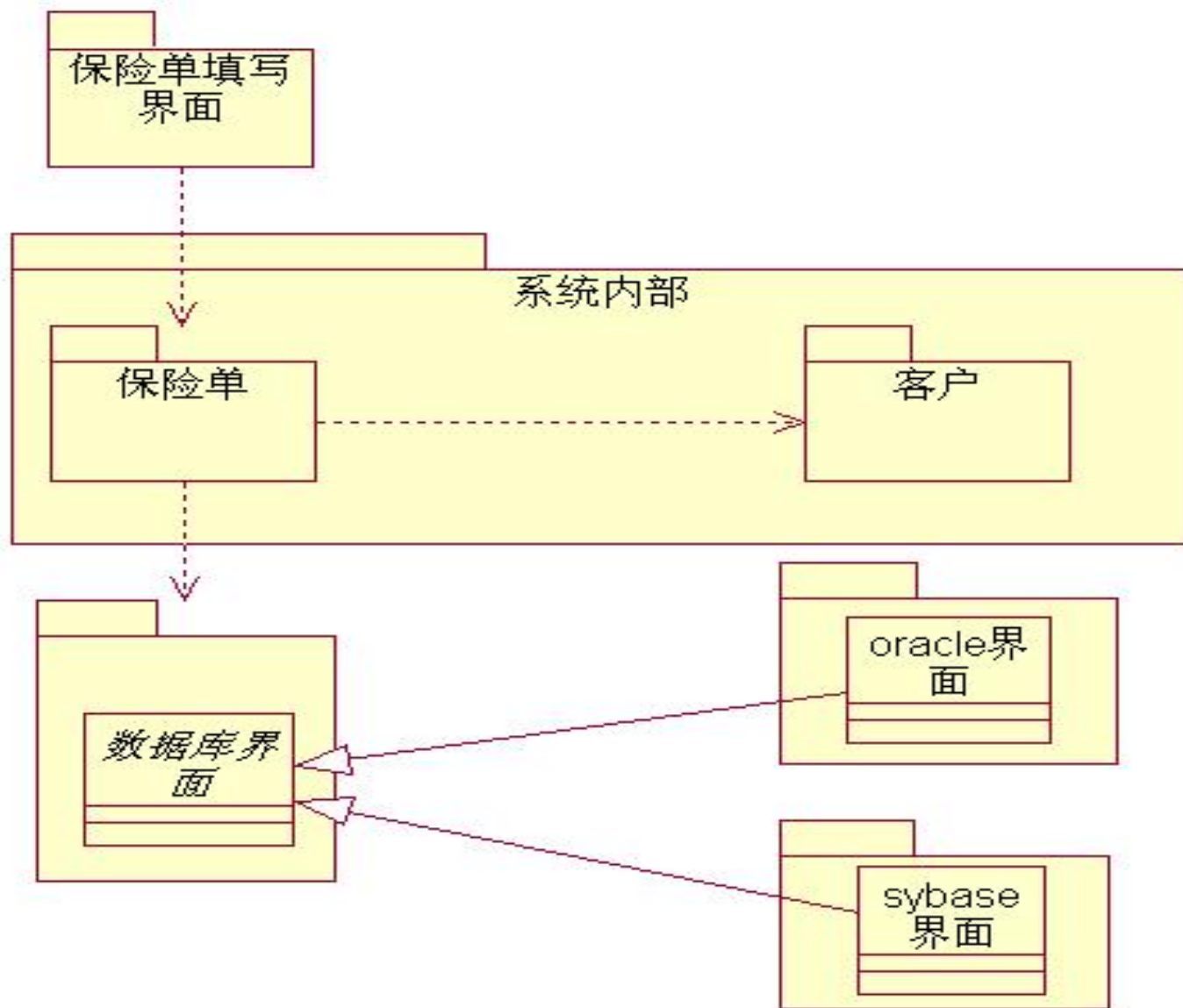


简单名



简单名

系统建模—5.包图



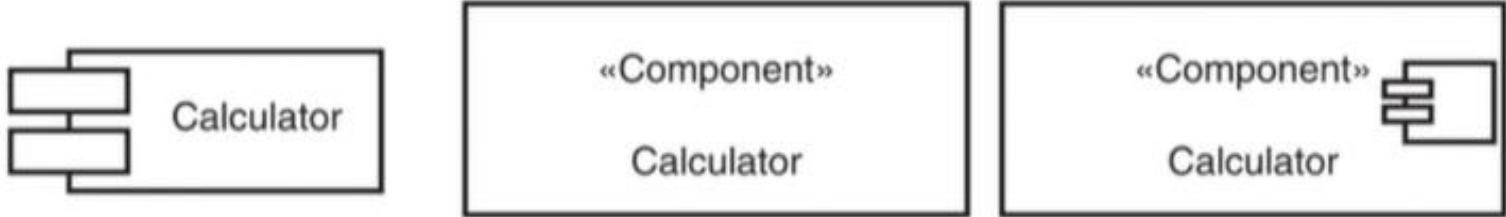
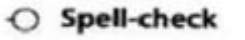
系统建模—5.构件图

UML中的物理实现图包括构件图和部署图两种类型。

构件可以包括源代码文件、二进制文件、脚本、可执行文件。

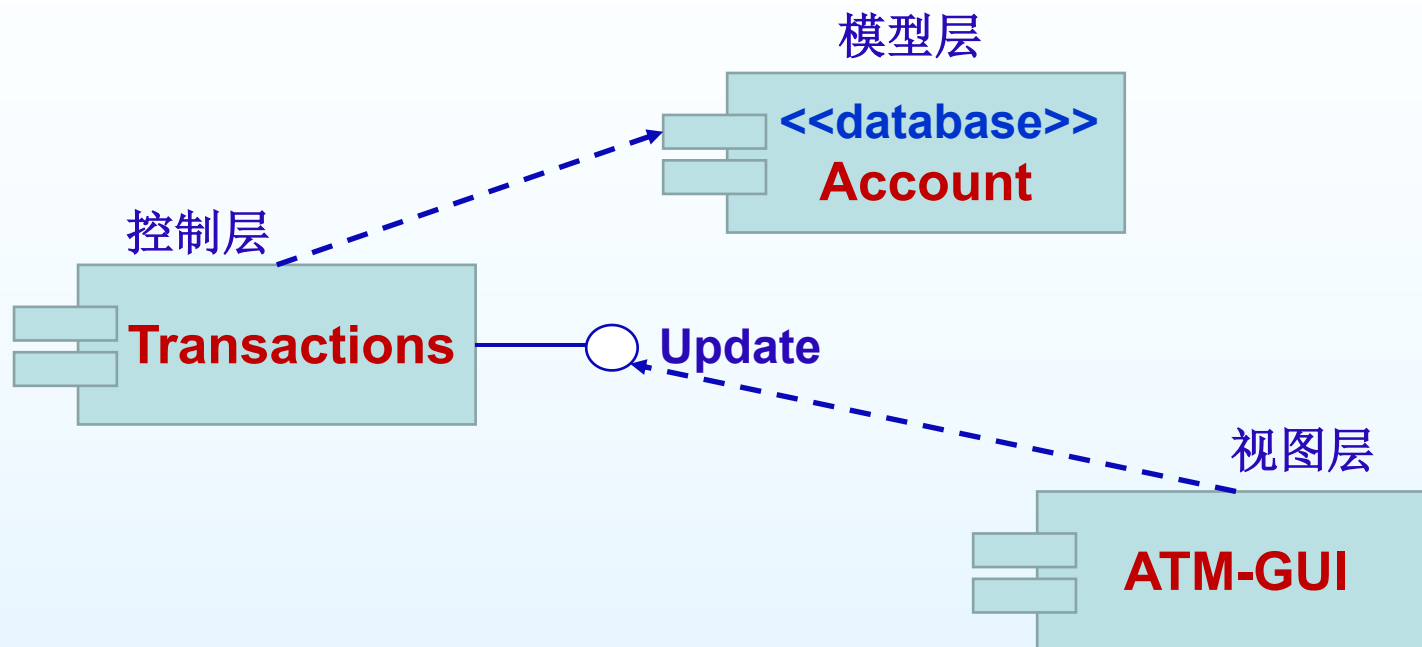
构件图用于静态建模，是表示构件类型的组织以及各种构件之间依赖关系的图。构件图通过对构件间依赖关系的描述来估计对系统构件的修改给系统可能带来的影响。

□

构件		
接口	外部可访问到的服务	
构件实例	节点实例上构件的一个实例，冒号后是该构件实例名称	
实现关系	构件向外提供的服务	
依赖关系	构件依赖外部提供的服务（由构件到接口）	

系统建模—5.构件图

□ATM机系统的组件图



构件是作为一个或多个类的软件实现，驻留在计算机中，提供和其他构件之间的接口，是系统中可替换的物理部件。

系统建模—6.部署图（配置图）

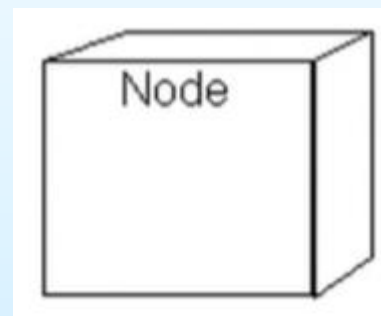
部署图用于静态建模，是表示运行时过程节点结构、构件实例及其对象结构的图。**可以帮助系统的有关人员了解软件中各个组件驻留在什么硬件上，以及这些硬件之间的关系。**

如果含有依赖关系的构件实例放置在不同节点上，部署视图可以展示出执行过程的瓶颈。

部署图通常包含2个元素：节点(Node)和关联关系(Association)。

(1) 节点

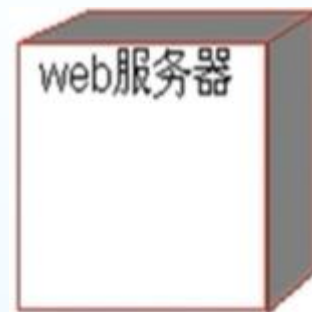
节点是在运行时代表计算资源的物理元素，表示运行软件的硬件单元，节点通常有一些内存，并具有处理能力，节点用一个立方体来表示。



系统建模—6.部署图（配置图）

在实际的建模过程中，可以把节点分为处理器(Processor)和设备(Device)两类。

□ 处理器(Processor): 能够执行软件、具有计算能力的节点。如：PC机、服务器、工作站等都属于处理器。



□ 设备(Device): 设备是没有计算能力的节点，通常情况下都是通过其接口为外部提供某种服务，例如：打印机、扫描仪、路由器等。



系统建模—6.部署图（配置图）

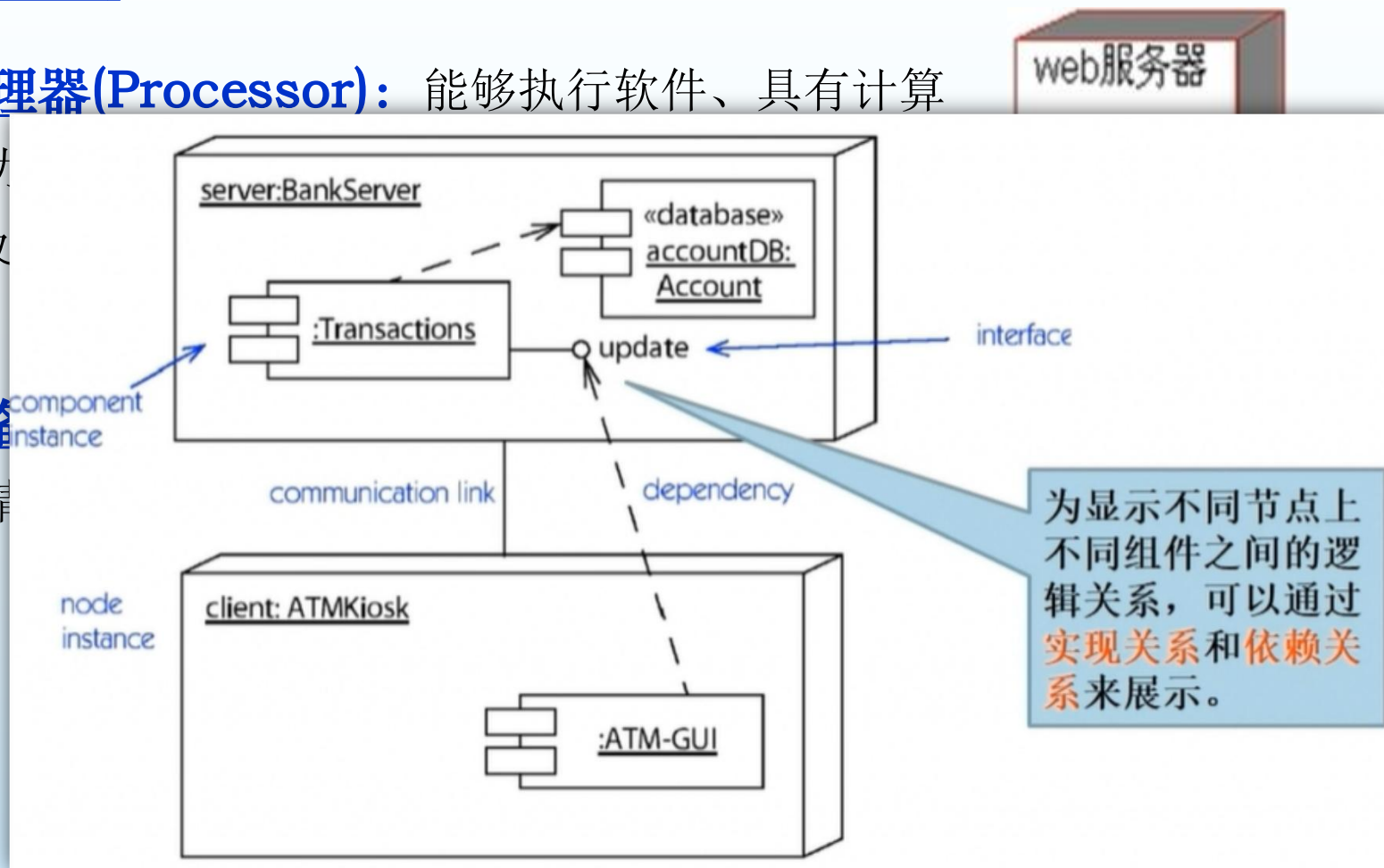
在实际的建模过程中，可以把节点分为处理器(Processor)和设备(Device)两类。

□ 处理器(Processor)：能够执行软件、具有计算

能力
于处

□ 设备

常情
如：



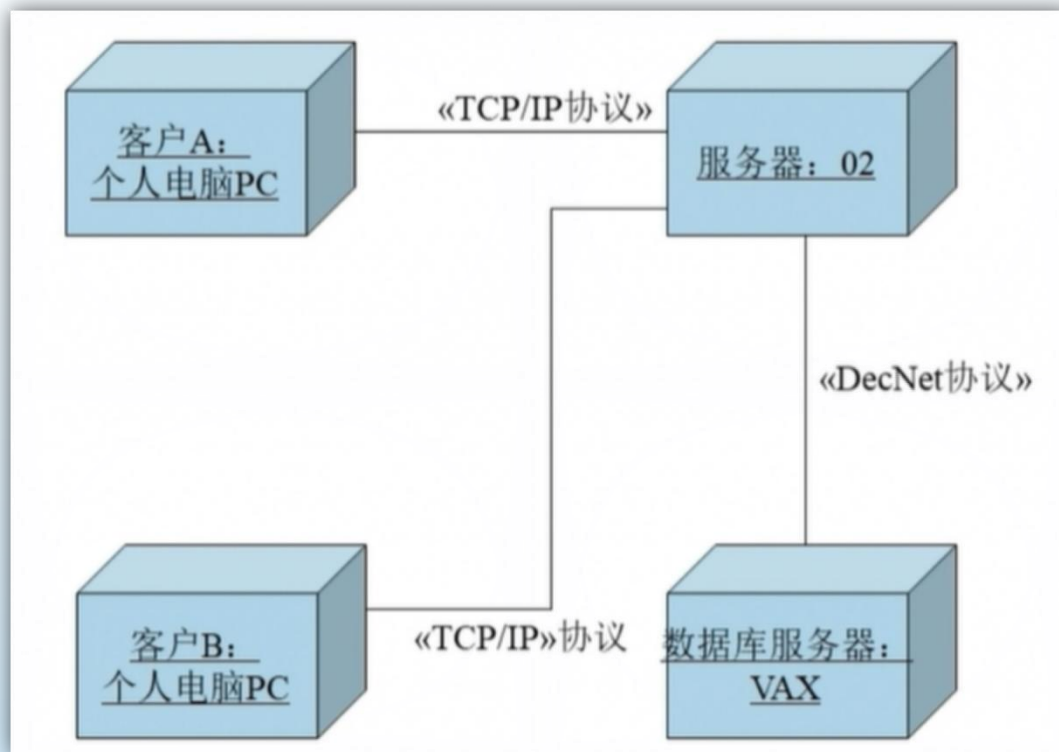
四、结构模型

6. 部署图（配置图）

(2) 关联关系

配置图用关联关系表示各节点之间通信路径，表示为一条实线。

在连接硬件时通常关心节点之间的连接方式（以太网、局域网、并行、**TCP**、**USB**等），因此关联关系一般不使用名称，而是使用构造型来表示连接方式，如<<Ethernet>>、<<local>>、<<TCP>>、<<HTTP>>等。



四、结构模型

6. 部署图（配置图）

(2) 关联关系

配置图用关联关系表示各节点之间通信路径，表示为一条实线。

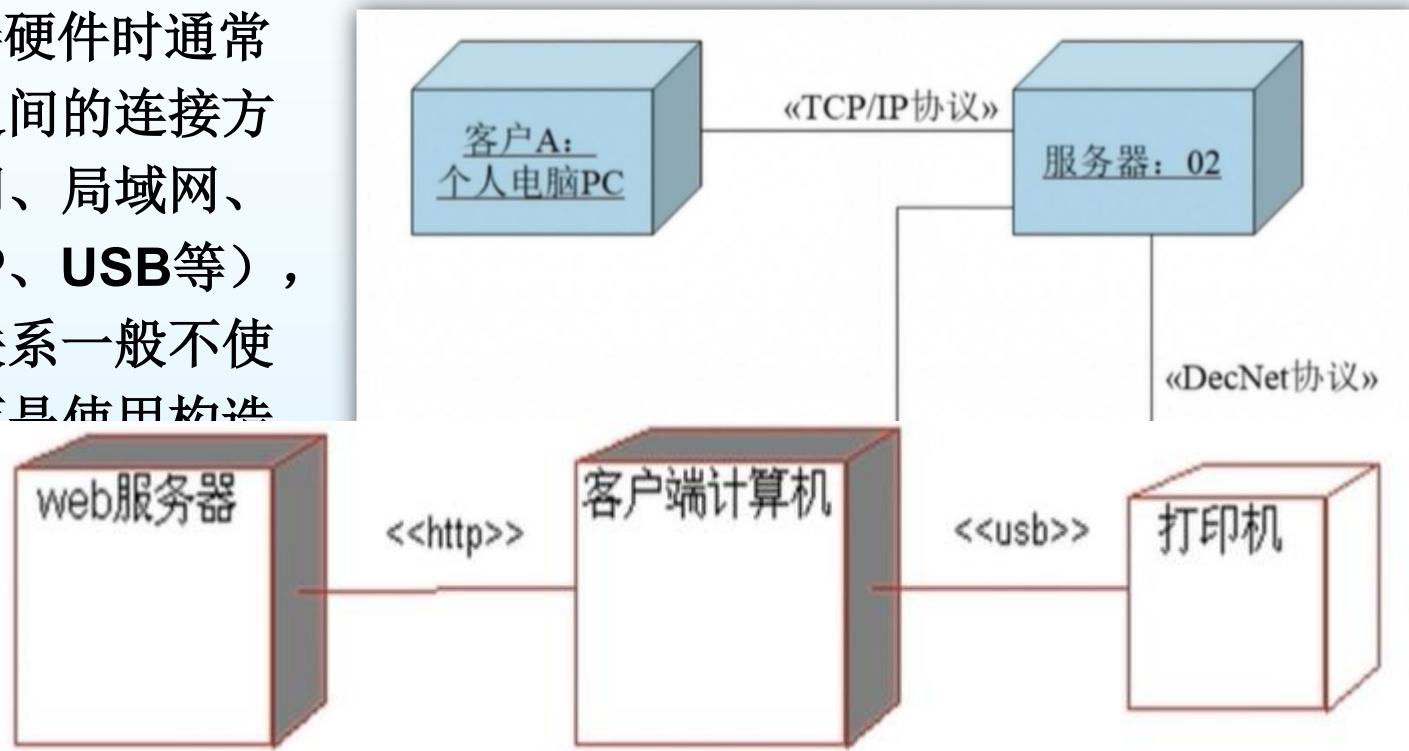
在连接硬件时通常关心节点之间的连接方式（以太网、局域网、并行、TCP、USB等），因此关联关系一般不使用名称，而且使用构造

型来表示：

<<Ethernet>>

<<local>>

<<HTTP>>



谢谢观看

